

BE : RÉDUCTION EN DIMENSION ET SEGMENTATION D'IMAGE

Consignes Générales :

- Par groupe de 4 à 5 étudiants maximum, vous devez rédiger un rapport dans lequel vous répondrez aux questions du sujet et plus globalement au problème demandé. La qualité de rédaction ainsi que tous les compléments que vous apporterez (réflexions, analyses des réponses, tests sur d'autres données, etc...) seront fortement pris en compte. Ce rapport devra être rédigé sous un traitement de texte (les pages manuscrites ne sont pas acceptées) et rendu sous la forme d'un fichier .pdf.
- Toutes les fonctions programmées le seront en Python et seront abondamment commentées. Le rendu du projet se fera sous la forme d'un fichier .zip contenant le rapport ainsi que l'ensemble des programmes et données nécessaires à l'exécution des programmes.
- Les fonctions programmées doivent l'être dans un fichier librairie ayant pour nom : **Ma324_BE_lib.py**. Dans ce fichier les différentes fonctions doivent être regroupées dans des blocs (commande : `#%%`) correspondants aux parties du sujet.
- Les applications numériques des fonctions du sujet seront dans un fichier ayant pour nom : **Ma324_BE_main.py**. Dans ce fichier les différents cas d'étude doivent être regroupés dans des blocs séparés.
- Les différentes données utilisées devront être enregistrées dans un dossier ayant pour nom **Data**. Ce dossier doit être mis dans le même répertoire que vos fichiers **Ma324_BE_lib.py** et **Ma324_BE_main.py**.
- Les noms des différents membres du groupe doivent apparaître en commentaire en début de fichier.
- Le non-respect des consignes précédentes entraînera des points de pénalité pouvant être conséquents...
- Concernant Python, le rendu attendu est l'utilisation des fonctions de bases des librairies Numpy, Matplotlib, Sympy et OpenCV. Néanmoins l'utilisation de fonctions plus avancées qui améliorent les rendus graphiques par exemple et/ou la programmation en terme de classe seront bien évidemment acceptées et seront valorisées.
- Les questions d'analyse de résultats sont généralement ouvertes. La qualité de vos réflexions sera fortement prise en compte pour la notation de ces questions.
- Il est impératif de lire le sujet en entier.
- Enfin on rappelle qu'un BE est un examen et comme tout examen il arrive de ne pas pouvoir répondre à toutes les questions.

Le but principal de ce BE est de créer un algorithme de **partitionnement** ou **segmentation d'image** (clustering) qui consistera dans ses grandes lignes à :

1. choisir des points au hasard sur une image : position i, j en lignes et colonnes sur l'image.
2. Pour chaque pixel (i, j) on associera un vecteur (ligne) contenant la position i, j ainsi que l'intensité des trois couches de couleurs mais également d'autres paramètres que vous définirez par la suite.
3. L'étape précédente permet de créer une matrice où chaque ligne représente les pixels choisis. On réalisera ensuite une réduction en dimension 2 sur ce tableau soit par une ACP classique ou par un auto-encodeur. On obtient une représentation 2D des pixels considérés avec leurs attributs (positions, couleurs, etc...).
4. Utiliser l'algorithme des k-moyennes pour créer des catégories/partitions entre les pixels. On associera à chaque catégorie une couleur afin de créer un « masque ponctuel » sur l'image.
5. On utilisera enfin l'algorithme d'inpainting, soit en réutilisant l'algorithme du BE de Ma321, soit celui d'OpenCV. Cet algorithme permettra de donner une couleur de catégorie à tous les pixels de l'image (et non uniquement aux pixels choisis initialement). On crée ainsi des masques sur l'image totale.

On obtient ainsi une segmentation de l'image.

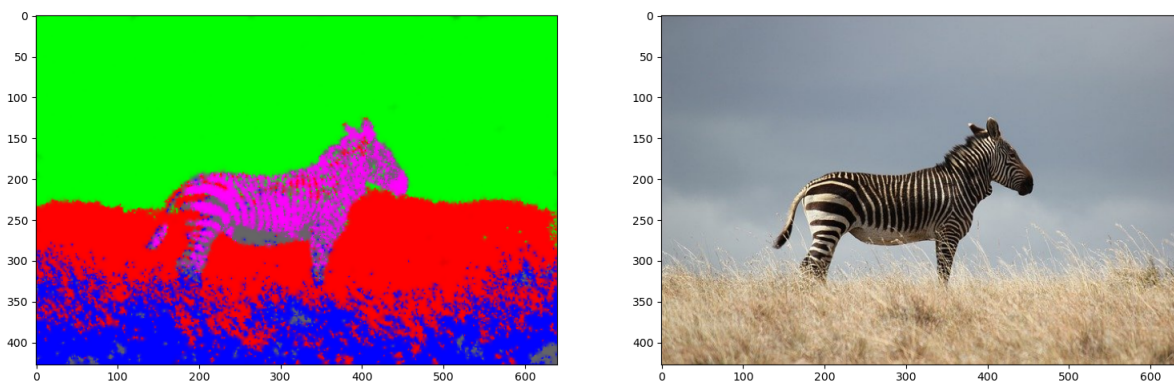


FIGURE 1 – Un exemple de segmentation d'image.

Partie 1 : L'ACP le retour du retour du retour...

Le but de notre future application de l'ACP est de le faire sur les pixels d'une image. Pour donner un ordre de grandeur, une petite image disons de 480 par 640 pixels contient : 307200 pixels... Comme présentée dans l'introduction, nous allons devoir choisir (au hasard) un certain nombre de points. Néanmoins même si nous choisissons 10 000 pixels, réaliser une ACP en utilisant la décomposition SVD est impossible. Dans le cas de 10 000 points la matrice U serait de taille 10 000 par 10 000.... grandeur qui dépasse l'utilisation mémoire autorisée d'un PC standard. Nous allons devoir donner une formulation moins gourmandes en espace mémoire de l'ACP. Il pourra être utile de reprendre l'ACP vu en Ma313 et Ma315 (fascicule de cours).

Considérons un tableau de données de la forme :

	X_1	X_2	$\cdots$	X_d
ind_1	x_{11}	x_{12}	$\cdots$	x_{1d}
ind_2	x_{21}	x_{22}	$\cdots$	x_{2d}
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
ind_n	x_{n1}	x_{n2}	$\cdots$	x_{nd}

On notera la matrice $R \in \mathcal{M}_{n,d}(\mathbb{R})$, la matrice associée à ces données. Chaque ligne (la ligne i sera notée ind_i) représente les données d'un individus et les colonnes (notée X_j) un type de donnée « mesurée » (i.e. des valeurs/réalisations d'une variable aléatoire). Notons :

$$ind_g := \frac{1}{n} \sum_{i=1}^n ind_i$$

le vecteur ligne centre de gravité des données et posons la nouvelle matrice centrée :

$$R_c := \begin{pmatrix} ind_1 - ind_g \\ ind_2 - ind_g \\ \vdots \\ ind_n - ind_g \end{pmatrix}$$

et l'on « réduit » la matrice en divisant chaque colonne de R_c par sa norme. Notons R_{cr} la forme centrée-réduite des données R .

On rappelle que pour un vecteur aléatoire $X := (X_1, \dots, X_d)$, on définit la **première composante principale** comme le vecteur aléatoire :

$$Y_1 := Xv_1 = \sum_{i=1}^n v_{1i}X_i$$

où v_1 est un vecteur de $\mathbb{R}^n$ de norme égal à 1 (i.e. appartenant à la sphère unité) et tel que :

$$v_1 = \arg \max_{\|v\|_2=1, v \in \mathbb{R}^d} Var(Xv)$$

On appelle v_1 la **première direction principale**. Par récurrence on définit la $(k+1)$ -**composante principale de X** de la manière suivante. Supposons que les k -premières composantes principales $Y_1, Y_2, \dots, Y_k$ sont connues (construites). On cherche alors un vecteur aléatoire :

$$Y_{k+1} := Xv_{k+1} = \sum_{i=1}^n v_{(k+1)i}X_i$$

où v_{k+1} est un vecteur de $\mathbb{R}^n$ de norme égale à 1 (i.e. appartient à la sphère unité) et tel que :

$$v_{k+1} = \arg \max_{\substack{v \perp Vec(v_1, v_2, \dots, v_k) \\ \|v\|_2=1, v \in \mathbb{R}^d}} Var(Xv)$$

avec $v \perp Vec(v_1, v_2, \dots, v_k)$ signifiant que v appartient à l'orthogonal de l'espace vectoriel engendré par les vecteurs $v_1, v_2, \dots, v_k$.

On rappelle que :

$$Var(Xv) = v^T Cov(X)v$$

et par suite en utilisant la réalisation R_{cr} , on définit alors **l'estimateur de la matrice de covariance** par :

$$C := R_{cr}^T R_{cr} \in \mathcal{M}_{d,d}(\mathbb{R})$$

Ainsi nous sommes amenés à résoudre les problèmes suivants :

$$v_{k+1} = \arg \max_{\substack{v \perp \text{Vec}(v_1, v_2, \dots, v_k) \\ \|v\|_2=1, v \in \mathbb{R}^d}} v^T R_{cr}^T R_{cr} v$$

1. En utilisant la méthode des multiplicateurs de Lagrange, justifier que les directions principales sont données par les vecteurs propres de C . Rappeler pourquoi la matrice C se diagonalise sous la forme :

$$C = V D V^T$$

où V est une matrice orthogonale. **On supposera à présent dans les valeurs propres de C sont rangées par ordre décroissant dans D .** Exprimer la variance en chacune des directions principales en fonction des éléments de D .

2. En justifiant qu'une SVD possible de R_{cr} s'écrit sous la forme : $R_{cr} = U \Sigma V^T$ avec la matrice V précédemment définie, justifier que les k premières composantes principales des réalisations sont données par :

$$R_{cr} V_{[k]} = \begin{pmatrix} \sigma_1 u_1 & \sigma_2 u_2 & \cdots & \sigma_k u_k \end{pmatrix}$$

où $V_{[k]}$ est la matrice de taille (d, k) formée des k -premiers vecteurs colonnes de V , u_i note les vecteurs colonnes de U et σ_i les valeurs singulières de R_{cr} . On retrouve la formule vue en Ma313 et Ma315. Or on voit qu'il n'est pas nécessaire de calculer U , seul V suffit.

3. On rappelle que la diagonalisation de C est obtenue en réalisant une SVD de C sous Python, on pourra utiliser l'option « hermitian=True » de la fonction `np.linalg.svd`. Rédiger une fonction Python : **ACP(X,k)** réalisant l'ACP d'un tableau de donnée R_{cr} en utilisant la diagonalisation de C (et non la SVD de R_{cr}) et renvoyant la matrice $R_{cr} V_{[k]}$. Dans la pratique on pourra prendre $k = 2$ ou bien la valeur de k donnée par la règle de Kaiser donnée sous la forme : k est le plus petit des indices tels que :

$$\lambda_k \geq \frac{1}{d} \sum_{\lambda \in \text{Spec}(C)} \lambda$$

4. Pour tester la robustesse de votre algorithme exécutez le sur les données déjà rencontrées en TDs : **iris.csv** et **mnist_train.csv**, par exemple. Pourquoi la réalisation d'une ACP en utilisant la SVD de la matrice R_{cr} n'est-elle pas possible pour des bases de données comme mnist ?
5. Tester une l'ACP pour $k = 1$ sur les données. Que remarque t-on ?
6. Certaines variables peuvent avoir une plus grande importance que d'autres, comme par exemple on peut vouloir que la position (i, j) des pixels soit plus valorisée que leurs couleurs. On peut alors introduire une matrice diagonale :

$$\sqrt{W} := \begin{pmatrix} \sqrt{w_1} & & & \\ & \sqrt{w_1} & & \\ & & \ddots & \\ & & & \sqrt{w_d} \end{pmatrix} \in \mathcal{M}_d(\mathbb{R})$$

et modifier la matrice de covariance par une version pondérée :

$$C_W = \frac{1}{\sum_{i=1}^d w_i} \left(X_{cr} \sqrt{W} \right)^T X_{cr} \sqrt{W}$$

Rédiger une fonction Python : **ACPpond(X,k,W)** réalisant l'ACP pondérée par une matrice $W = \sqrt{W}^2$. Tester en changeant la matrice W le résultat obtenue avec la base de donnée **iris.csv** puis **mnist_train.csv**.

Partie 2 : γνῶθι σεαυτόν¹

Dans cette partie nous allons aborder le problème de la réduction dimensionnelle des données en utilisant cette fois les réseaux de neurone tels que nous les avons introduit lors du TD4 . La stratégie sera de faire apprendre à notre réseau de neurone la fonction identité². Pour ce faire nous allons imposer que notre réseau de neurones admette la structure suivante :

- le nombre de couche L doit être un nombre impair i.e. : $L = 2q - 1$, où $q \geq 2$.
- et la taille de ces couches doit vérifier la relation suivante :

$$n_{i+1} = n_{L-i}$$

pour $i \in \llbracket 0, q-1 \rrbracket$.

On appelle ce type de réseau de neurones un **auto-encodeur**.

Nous reprenons ici les notations principales du TD4. On suppose fixée une fonction d'activation. Néanmoins comme nous allons centrer et réduire les données d'apprentissage nous allons choisir des fonctions d'activations dont l'image est dans l'intervalle $[-1, 1]$. Par exemple on pourra choisir la fonction tangente hyperbolique :

$$\sigma(x) := \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$$

Le type de réseau de neurones (multicouches) que nous allons utiliser sera encore notée :

$$a^{(L)} : \mathbb{R}^{n_1} \rightarrow \mathbb{R}^{n_L}$$

mais cette fois avec $n_1 = n_L$. La structure imposée précédemment nous permet d'introduire (toujours avec les notations du TD) les deux fonctions suivantes :

- la fonction d'encodage $\mathcal{E}$ définie par :

$$\mathcal{E} := a^{(q)} : \mathbb{R}^{n_1} \rightarrow \mathbb{R}^{n_q}$$

- et la fonction de décodage $\mathcal{D}$ la fonction composée :

$$\mathcal{D} := \mathbb{R}^{n_q} \rightarrow \mathbb{R}^{n_L}$$

définie pour tout $y \in \mathbb{R}^{n_q}$ par la relation de récurrence :

$$\begin{cases} \mathcal{D}^{(q)}(y) := y \\ \mathcal{D}^{(j)}(y) := \sigma \odot \left(W^{(j)} \mathcal{D}^{(j-1)}(y) + b^{(j)} \right) & j \in \llbracket q+1, L \rrbracket \\ \mathcal{D}(y) = \mathcal{D}^{(L)}(y) \end{cases}$$

Ces deux fonctions vérifient ainsi :

$$a^{(L)} = \mathcal{D} \circ \mathcal{E}$$

1. Locution grecque signifiant : « connais-toi toi même ».

2. En réalité ce n'est pas exactement la fonction identité que l'on apprend mais plutôt « l'identité sur les exemples d'entraînement ». Néanmoins cette formulation est plus « marquante » dit comme cela et permet de justifier le titre choisi de cette partie...

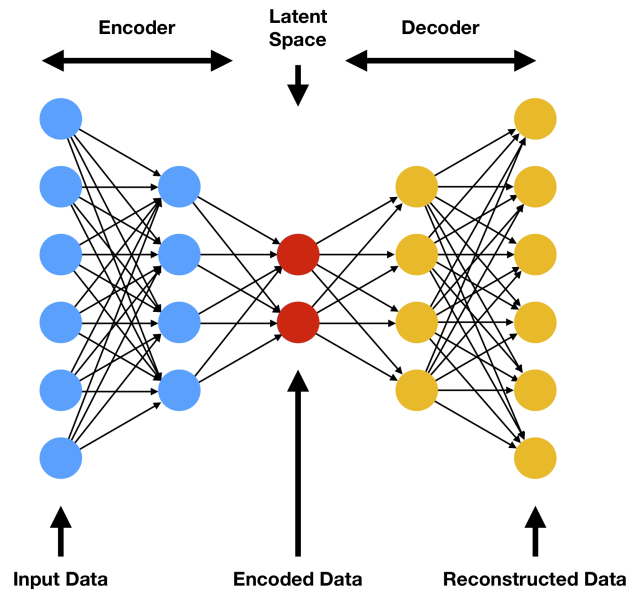


FIGURE 2 – Dans cet exemple on a un réseau à $L = 5$ couches. Dans le détail on a : $n_1 = n_5 = 6$ (même taille des entrées et des sorties), $n_2 = n_4 = 4$ et $n_3 = 2$. (source image : <https://irma.math.unistra.fr/~franck/cours/SciML/output/html/chapAPsec5.html>)

Les **données d'entraînements** seront dans le cas de l'auto-encodeur les couples $(x^{(i)}, x^{(i)})$ pour $i \in \llbracket 1, N \rrbracket$ où N est un entier (généralement grand) et où : $x^{(i)} \in \mathbb{R}^{n_1}$. Pour ce BE nous conserverons la **fonction coût** suivante :

$$\arg \min_{W^{(j)}, b^{(j)}} \frac{1}{N} \sum_{i=1}^N \frac{1}{2} \|x^{(i)} - a^{(L)}(x^{(i)})\|_2^2$$

Autrement dit nous cherchons les poids du réseau de sorte à avoir :

$$a^{(L)}(x^{(i)}) = x^{(i)}$$

pour tout $i \in \llbracket 1, N \rrbracket$.

On rappelle que l'algorithme d'apprentissage des poids est donné par :

Algorithme 1 Apprentissage d'un auto-encodeur avec l'algorithme du gradient stochastique.

Fixer $L > 2$ et pour tout $i \in \llbracket 1, L \rrbracket$ **fixer** un entier n_i .

Fixer $\eta > 0$ (appelé le taux d'apprentissage).

Choisir (aléatoirement) des matrices $W^{(j)} \in \mathcal{M}_{n_j, n_{j-1}}(\mathbb{R})$ et des vecteurs $b^{(j)} \in \mathbb{R}^{n_j}$ pour tout $j \in \llbracket 2, L \rrbracket$.

Pour un nombre d'itération fixé Nbiter :

Choisir aléatoirement $i \in \llbracket 1, N \rrbracket$ un exemple d'entraînement : $x^{(i)}$

$a^{(1)} = x^{(i)}$

Pour j de 2 à L :

$$z^{(j)} = W^{(j)} a^{(j-1)} + b^{(j)}$$

$$a^{(j)} = \sigma \odot (z^{(j)})$$

$$\delta^{(L)} = \sigma' \odot (z^{(L)}) \odot (a^{(L)} - x^{(i)})$$

Pour j de $L-1$ à 2 :

$$\delta^{(j)} = \sigma' \odot (z^{(j)}) \odot \left((W^{(j+1)})^T \delta^{(j+1)} \right)$$

Pour j de L à 2 :

$$W^{(j)} \leftarrow W^{(j)} - \eta \delta^{(j)} (a^{(j-1)})^T$$

$$b^{(j)} \leftarrow b^{(j)} - \eta \delta^{(j)}$$

1. Si cela n'est pas encore fait programmer les fonctions demandées en TD : **init_RN(Ln)**, **train_RN(Ln, train_data, Y, W, b, eta, Nbiter)** et **cout(train_data, Y, W, b)**. Afin d'accélérer les calculs on peut ne pas afficher la valeur de la fonction coût à chaque itération (par exemple toutes les 1000 ou 10 000 itérations). L'entraînement d'un réseau de neurones peut être fait en « plusieurs fois » en enregistrant les listes contenant les poids et en les chargeant pour les utiliser comme initialisation d'un nouvel entraînement. On donne par exemple les commandes suivantes :

```
np.save("W.npy", np.array(W, dtype=object), allow_pickle=True)
np.save("b.npy", np.array(b, dtype=object), allow_pickle=True)
#pour le chargement
W = np.load('W.npy', allow_pickle=True)
b = np.load('b.npy', allow_pickle=True)
```

2. Implémenter les fonctions d'encodage $\mathcal{E}$ et de décodage $\mathcal{D}$.
3. Tester votre fonction d'encodage pour $n_q = 2$ en faisant varier les couches de l'auto-encodeur ainsi que les fonctions d'activations sur la base de donnée **iris.csv** puis **mnist_train.csv**.



FIGURE 3 – Exemple de sortie sur Iris pour l'ACP, en haut, et l'auto-encodeur (fonction d'encodage) en bas. Ici $n_1 = n_7 = 4$, $n_2 = n_4 = n_3 = n_6 = 3$ et $n_5 = 2$. On a pris $\eta = 0.0001$ et $Nbiter = 10\,000\,000$. La fonction d'activation est la fonction tangente hyperbolique. On a fait apparaître en couleurs les catégories réelles.

4. L'intérêt de l'approche par auto-encodeur est de fournir également une fonction de décodage. Cette fonction de décodage peut servir à générer des données à partir d'un espace de dimension plus petit car n_L est bien plus grand que n_q . En appliquant ceci sur **mnist_train.csv** utiliser la fonction de décodage afin de générer de nouvelles images.

Partie 3 : L'algorithme de segmentation.

Nous allons programmer sous forme de fonctions séparées chacun des points annoncés en début de BE. Néanmoins lorsque vous serez assurés du bon fonctionnement de vos fonctions vous pouvez créer (cela est même conseillé) une fonction globale réalisant la tâche de segmentation d'une image.

1. Créer une fonction **choixpts(img, nbpts)** qui à partir d'une image **img** (couleur) et d'un nombre de points **nbpts** renvoie une liste L de nbpts choisis au hasard en suivant une loi uniforme sur les lignes, sauf la première et la dernière, de même sur les colonnes sauf la première et la dernière.
2. Une fonction **data_pixels(img, L)** qui prend en entrée une image **img** et **L** la sortie de la fonction précédente et qui renvoie une matrice de taille nbpts×8 et où les lignes sont données par :

$$(c_{i1}, c_{i2}, c_{i3}, c_{i4}, c_{i5}, c_{i6}, c_{i7}, c_{i8})$$

avec :

- c_{i1} l'indice de ligne du pixel i , c_{i2} l'indice de colonne,
- c_{i3}, c_{i4}, c_{i5} les trois intensités de couleurs sur les trois couches de l'image,
- c_{i6}, c_{i7}, c_{i8} les trois moyennes des intensités de couleurs sur les 9 pixels autour du pixel i (lui compris).

On notera **data_img** la matrice de sortie précédente.

3. Une fonction **ACPimg(data_img)** qui réalise une ACP avec $q = 2$ des données **data_img**. De même une fonction **REDimg(data_img)** qui réalise une réduction de dimension grâce à un auto-encodeur avec $q = 2$.
4. Cette étape comportera deux fonctions.
 - (a) Une fonction **Kmoyimg(data_img, qkayser)** qui réalise un partitionnement à l'aide de l'algorithme des k-moyennes pour un k que vous choisirez ou on pourra prendre le k donné par la règle de Kaiser lors de l'ACP. Comme pour le BE de Ma313 on pourra utiliser l'algorithme des k-moyennes de sklearn.
 - (b) Une fonction **Masque(S)** qui prend en entrée la matrice de sortie de l'algorithme S et qui crée en sortie une image de même taille que **img**, l'image de départ, dont les pixels sont :
 - soit noir si le pixel (i, j) n'est pas dans les pixels choisis initialement,
 - soit d'une couleur différente pour chaque catégorie produite par l'algorithme des k-moyennes.

On notera **imgmasqueponctuel** par la suite cette image de sortie.

5. On utilisera l'algorithme d'inpainting³ (ou la version programmée en Ma321). On pourra utiliser les commandes suivantes que l'on encapsulera dans une fonction : **RemplissageMasque(imgmasqueponctuel)** où **imgmasqueponctuel** est l'image représentant le masque ponctuel précédent :

```
mask=cv2.cvtColor(imgmasqueponctuel, cv2.COLOR_BGR2GRAY)
mask[mask>0]=255
mask=255*np.ones(np.shape(mask))-mask
imgmasque = cv2.inpaint(imgmasqueponctuel, np.uint8(mask), 3, cv2.INPAINT_NS)
```

Cette fonction renverra en sortie l'image **imgmasque**. On pourra afficher cette image grâce à la commande `cv2.imshow()` afin de voir apparaître les différentes zones trouvées par l'algorithme. On pourra également afficher les zones de l'image originelle dans des images séparées.

6. Tester votre algorithme sur des images et comparer les différents résultats obtenus entre la version ACP et auto-encodeur.
7. Modifier l'algorithme précédent en remplaçant l'ACP classique par une ACP pondérée. Commenter les résultats finaux en fonction des poids et des images utilisés.

En guise d'exemple, considérons l'image suivante :

3. cf. https://docs.opencv.org/3.4/df/d3d/tutorial_py_inpainting.html



FIGURE 4 – Image originelle

Après avoir choisi 10 000 pixels au hasard de manière uniforme puis réaliser l'ACP sur la matrice `data_img`, on trouve ici `qkayser=2`, on obtient alors le masque ponctuel et le masque global suivants :



FIGURE 5 – Masque ponctuel (à gauche), masque après inpainting (à droite)

En utilisant le masque global on obtient enfin le résultat suivant sur l'image de base :

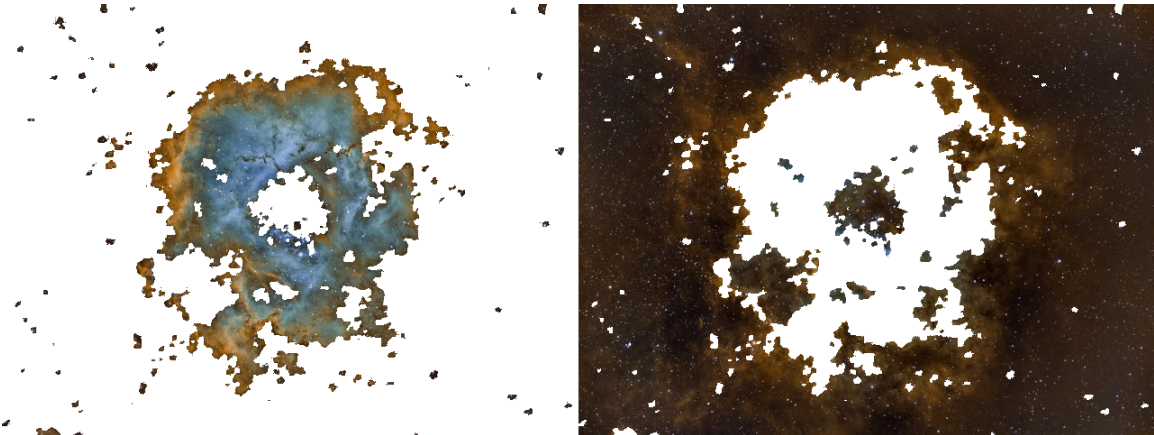


FIGURE 6 – Rendu final avec les couches séparées.